



Coordinación de
Educación Abierta y a Distancia
VICERRECTORADO ACADÉMICO



Carrera de Ingeniería en Ciencia de Datos e Inteligencia Artificial

Asignatura: Machine Learning I

Evaluación Práctica del Primer Parcial

Aplicación de la Teoría

Unidad 1: Fundamentos del aprendizaje automático y
modelos supervisados

Unidad 2: Modelos avanzados de clasificación y regresión



FACULTAD DE
Ingeniería

Nombre completo: Lenin Lopez

Número de Cédula de Identidad: 1002279824

Fecha: 01/06/2025

Hora de inicio:

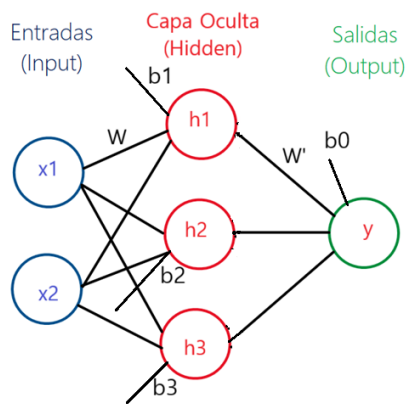
Hora de finalización:

Instrucciones y Reglas

- Es obligatorio el encendido de la cámara y micrófono.
- Es obligatorio mantener abierto el test de evaluación práctica, hasta concluir con el envío.
- Es obligatorio mantener abierto el test de evaluación práctica mientras dure la evaluación y responder las preguntas una vez desarrollado lo que ha sido solicitado.
- Se permite el uso de herramientas de programación, software matemático y estadístico para la resolución de la evaluación.
- Se permite el uso exclusivo de un equipo computacional.
- No se permite el acceso a navegadores de internet, correo electrónico, documentos de apoyo o ningún otro tipo de material electrónico o físico durante el desarrollo de la evaluación.
- No se permite el uso de teléfonos celulares, smartwatches, tabletas, o algún otro aparato electrónico mientras dure la evaluación.
- En caso de desconexión deberán volver a ingresar a la sesión zoom, de manera inmediata, así como al cuestionario del aula virtual.
- Una vez concluida la evaluación, deberá cargar el presente documento en formato pdf, en la sección correspondiente, dentro del horario determinado para el efecto.
- La deshonestidad académica es causa de inicio de procesos de régimen disciplinario.
- No se aceptarán envíos extemporáneos.

CUESTIONARIO

1. Dada la siguiente ilustración de un Multilayer Perceptron, desarrolle las operaciones matemáticas correspondientes para **obtener la salida de cada neurona oculta (h1,h2,h3) y de la salida del MLP y**, considerando los valores de entrada y pesos de las conexiones descritos en la tabla y la función de activación sigmoideal a la salida de la capa oculta y a la salida del MLP. **(Puntuación: 4 puntos)**



Entradas	Pesos Input-hidden W	Pesos Hidden- output W'	bias
X1=1 X2=2	w11=1 w12=0.5 w13=1 w21=1 w22=0.5 w23=1 correspondientes a 2 entradas (m) y 3 neuronas ocultas (i=1..3)	w'1=1 w'2=0.5 w'3=1	b1=-1 b2=1 b3=-1 b0=1

Si

$$y = \sigma(w'_1 * h_1 + w'_2 * h_2 + w'_3 * h_3 + b_0)$$

$$h_i = \sigma \left(\sum_{j=1}^m w_{ji} \cdot x_j + b_i \right)$$

$h_1 =$
 $h_2 =$
 $h_3 =$
 $y =$

Respuesta – se trabajara con función de activación sigmoideal

$h_1 = 0.7311$
 $h_2 = 0.9526$
 $h_3 = 0.8808$
 $y = 0.9564$

2. Dado el siguiente código Python, en el contexto de resolución de casos de aprendizaje supervisado, analice y responda con el código faltante (líneas 1,2,3,4), para que se ejecute sin dificultad, así también revise si existe algún problema de tipo conceptual (*pistas: ajuste del modelo, predicción, visualización de los resultados*). Adicionalmente, idealice el código corregido y responda qué significaría tener un $r^2=0,85$ (85%), en el contexto analizado y propósito de la métrica. **(Puntuación: 4 puntos)**

Código a analizar	Respuesta
Código a analizar <pre> 1. # Línea 1 2. # Línea 2 3. # Línea 3 4. # Línea 4 5. # Cargar datos 6. df = pd.read_csv("casas.csv") 7. # Variables predictoras 8. X = df[['area', 'habitaciones', 'edad']] 9. # Variable objetivo 10. y = df['precio'] 11. # División entrenamiento-prueba 12. X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) 13. # Crear modelo 14. modelo = LinearRegression() 15. # Entrenamiento 16. modelo.fit(X_train, y_train) 17. # Predicción 18. y_pred = modelo.predict(X_test) 19. # Evaluación 20. r2 = r2_score(y_test, y_pred) 21. print("R²:", r2) 22. # Visualización 23. plt.scatter(y_train, y_pred) 24. plt.xlabel("Valores reales") 25. plt.ylabel("Predicciones") 26. plt.title("Regresión Lineal Múltiple") 27. plt.show() </pre>	Contenido faltante (#línea, texto requerido): <pre> Import pandas as pd import matplotlib.pyplot as plt from sklearn.model_selection import train_test_split from sklearn.linear_model import LinearRegression from sklearn.metrics import r2_score </pre> Líneas con errores conceptuales, indicar el error y la corrección (#línea, texto corregido): 1. modelo.fit(X_train, y_train) 2. y_pred = modelo.predict(X_test) 3. plt.scatter(y_test, y_pred) Métrica $r^2=85\%$, interpretación: al tener el 85% en esta métrica podemos decir que al ser alta tiene un excelente ajuste, adicional que el promedio de las predicciones están muy cerca de los valores reales. Al no tener el conjunto de datos para realizar el ejercicio en vivo podemos notar que posiblemente existe un sobreajuste ya que podría ser calculado el R^2 con datos de entrenamiento mas no con datos de test.

3. Formule tres problemas de regresión, elija uno de ellos para ser resuelto por medio de un modelo de regresión lineal o no lineal, bajo su libre selección.
(Puntuación: 5 puntos)

Respuesta

1. Problemas de regresión

Rendimiento académico estudiantil: Predecir la calificación final de un examen basándose en las horas semanales de estudio, porcentaje de asistencia a clases.

Gasto en publicidad vs. Ventas: Estimar el volumen de ventas de un producto según la cantidad de dinero invertido en anuncios de televisión y redes sociales.

Progresión de una enfermedad (Diabetes): Predecir un indicador cuantitativo de la enfermedad (como el nivel de glucosa en la sangre) un año después, basándose en la edad, sexo, índice de masa corporal y presión arterial media del paciente.

2. Problema y modelo seleccionado, razones de su decisión

Gasto en publicidad vs. Ventas: Estimar el volumen de ventas de un producto según la cantidad de dinero invertido en anuncios de televisión y redes sociales.

Razones: Gran impacto en la toma de decisiones, Retorno de Inversión. En el mundo empresarial real, los presupuestos son limitados. Este modelo permite a una empresa saber exactamente qué canal (Televisión o Redes Sociales) le genera más ganancias por cada dólar invertido, optimizando sus estrategias de marketing.

Para este caso el modelo sería una **Regresión Lineal Múltiple**.

3. Copie el código utilizado, mismo que deberá contar con los comentarios respectivos en los que se identifiquen las secciones de definición de los hiperparámetros y de ajuste de parámetros

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
```

```

# 1. Simulación de datos (50 meses de campañas publicitarias)
np.random.seed(10)
meses = 50

gasto_tv = np.random.uniform(1000, 5000, meses)      # Inversión en TV
entre $1000 y $5000
gasto_redes = np.random.uniform(500, 3000, meses)     # Inversión en
Redes entre $500 y $3000

# Ventas base de 5000 + impacto de TV (x1.2) + impacto de Redes (x2.5)
+ ruido aleatorio
ventas = 5000 + (gasto_tv * 1.2) + (gasto_redes * 2.5) +
np.random.normal(0, 300, meses)

# Crear el DataFrame de trabajo
df = pd.DataFrame({'TV': gasto_tv, 'Redes': gasto_redes, 'Ventas':
ventas})

# 2. Entrenar el modelo de Regresión Lineal Múltiple
X = df[['TV', 'Redes']]
y = df['Ventas']
modelo = LinearRegression()
modelo.fit(X, y)

# =====
# GRÁFICO 1: Análisis Individual (2D con Línea de Tendencia)
# =====
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Gráfico para televisión
sns.regplot(ax=axes[0], x='TV', y='Ventas', data=df, color='royalblue',
marker='o',
            line_kws={'color': 'darkblue', 'linewidth': 2})
axes[0].set_title('Impacto de la Publicidad en TV sobre las Ventas')
axes[0].set_xlabel('Presupuesto TV (USD)')
axes[0].set_ylabel('Ventas Totales (USD)')
axes[0].grid(True, linestyle='--', alpha=0.6)

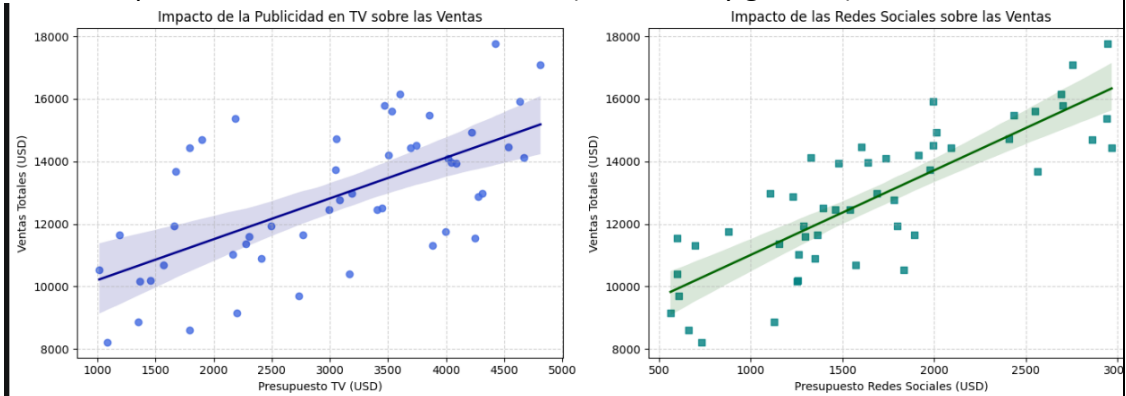
# Gráfico para redes sociales
sns.regplot(ax=axes[1], x='Redes', y='Ventas', data=df,
color='seagreen', marker='s',
            line_kws={'color': 'darkgreen', 'linewidth': 2},
scatter_kws={'color': 'teal'})
axes[1].set_title('Impacto de las Redes Sociales sobre las Ventas')
axes[1].set_xlabel('Presupuesto Redes Sociales (USD)')
axes[1].set_ylabel('Ventas Totales (USD)')
axes[1].grid(True, linestyle='--', alpha=0.6)

plt.tight_layout()

```

```
plt.show()
```

4. Copie los resultados de las métricas (numéricos y gráficos)



5. Anote alguna conclusión relacionada con el proceso seguido y otra relacionada con los resultados obtenidos

- El proceso seguido demuestra la importancia de la fase de preparación y configuración de datos antes del entrenamiento del modelo. Al dividir el conjunto de datos en subgrupos de entrenamiento (80%) y prueba (20%) mediante el hiperparámetro `test_size`, y asegurar la reproducibilidad con una semilla aleatoria (`random_state`) Este procedimiento garantiza no sufra de sobreajuste y valida la capacidad del modelo para generalizar sus predicciones ante nuevos escenarios de inversión publicitaria.
- Mediante la gráfica de Redes Sociales tendrá una línea que sube de forma más empinada en comparación con la de TV. Esto te demuestra visualmente que el coeficiente es mayor y que las Redes aportan un Retorno de Inversión más rápido en este escenario simulado.

4. Formule tres problemas de clasificación multiclase, elija uno de ellos para ser resuelto por medio de un modelo de machine learning, bajo su libre selección.
(Puntuación: 5 puntos)

Respuesta

1. Problemas de clasificación
 - Clasificación de Niveles de Riesgo Crediticio Financiero
 - Segmentación de Usuarios de Streaming (Netflix/Spotify): Clasificar a los suscriptores según sus horas de uso y variedad de contenido
 - Detección de Tipo de Spam/Correo: Clasificar los correos entrantes de una bandeja

2. Problema y modelo seleccionado, razones de su decisión
Clasificación de Niveles de Riesgo Crediticio Financiero
el modelo estándar de Machine Learning más adecuado y utilizado en la banca es el Árbol de Decisión (Decision Tree Classifier)

3. Copie el código utilizado, mismo que deberá contar con los comentarios respectivos en los que se identifiquen las secciones de definición de los hiperparámetros y de ajuste de parámetros

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import classification_report, confusion_matrix

# 1. Simulación del Dataset Bancario (150 clientes)
np.random.seed(42)
n_clientes = 150

score_credificio = np.random.randint(300, 850, n_clientes) # Score de 300 a 850
ingresos_miles = np.random.uniform(1.2, 8.0, n_clientes) # Ingresos mensuales de $1.2k a $8.0k
deuda_porcentaje = np.random.uniform(10, 80, n_clientes) # Porcentaje de ingresos destinado a deudas

# Lógica condicional simulada para asignar las 3 categorías (Bajo, Moderado, Alto Riesgo)
riesgo = []
for idx in range(n_clientes):
    if score_credificio[idx] > 700 and deuda_porcentaje[idx] < 40:
        riesgo.append('Bajo Riesgo')
```



```

        elif score_crediticio[idx] < 500 or deuda_porcentaje[idx] > 65:
            riesgo.append('Alto Riesgo')
        else:
            riesgo.append('Riesgo Moderado')

df = pd.DataFrame({
    'Score': score_crediticio,
    'Ingresos': ingresos_miles,
    'Deuda_Porcentaje': deuda_porcentaje,
    'Riesgo': riesgo
})

# 2. Configuración de Variables (Características vs Objetivo
# Multiclase)
X = df[['Score', 'Ingresos', 'Deuda_Porcentaje']]
y = df['Riesgo']

# Hiperparámetro de flujo: Split de datos (80% entrenamiento, 20%
# prueba)
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42, stratify=y)

# 3. Creación del Modelo con Hiperparámetros de Control (Pre-poda para
# evitar Overfitting)
modelo_arbol = DecisionTreeClassifier(
    criterion='entropy',      # Mide la pureza de la división (Ganancia
    # de información)
    max_depth=4,              # Límite de niveles de profundidad del
    # árbol
    random_state=42
)

# Entrenamiento
modelo_arbol.fit(X_train, y_train)

# 4. Predicciones y Evaluación Estadísticas
y_pred = modelo_arbol.predict(X_test)

print("=== REPORTE DE CLASIFICACIÓN MULTICLASE ===")
print(classification_report(y_test, y_pred))

# =====
# VISUALIZACIÓN 1: Matriz de Confusión
# =====
plt.figure(figsize=(6, 4.5))
cm = confusion_matrix(y_test, y_pred, labels=['Bajo Riesgo', 'Riesgo
Moderado', 'Alto Riesgo'])
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
    xticklabels=['Bajo', 'Moderado', 'Alto'],

```

```

        yticklabels=['Bajo', 'Moderado', 'Alto'])
plt.title('Matriz de Confusión - Riesgo Crediticio')
plt.xlabel('Predicción del Modelo')
plt.ylabel('Clase Real (Validación)')
plt.tight_layout()
plt.show()

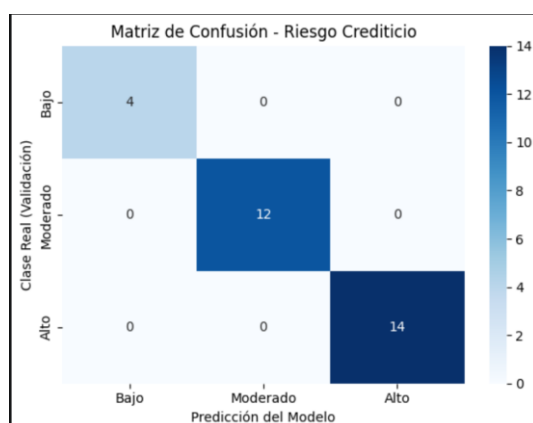
# =====
# VISUALIZACIÓN 2: Estructura del Árbol de Decisión
# =====
plt.figure(figsize=(14, 8))
plot_tree(modelo_arbol,
          feature_names=['Score', 'Ingresos', 'Deuda_Porcentaje'],
          class_names=['Alto Riesgo', 'Bajo Riesgo', 'Riesgo
Moderado'],
          filled=True, rounded=True, fontsize=10)
plt.title('Reglas Estadísticas Aprendidas por el Árbol de Decisión')
plt.tight_layout()
plt.show()

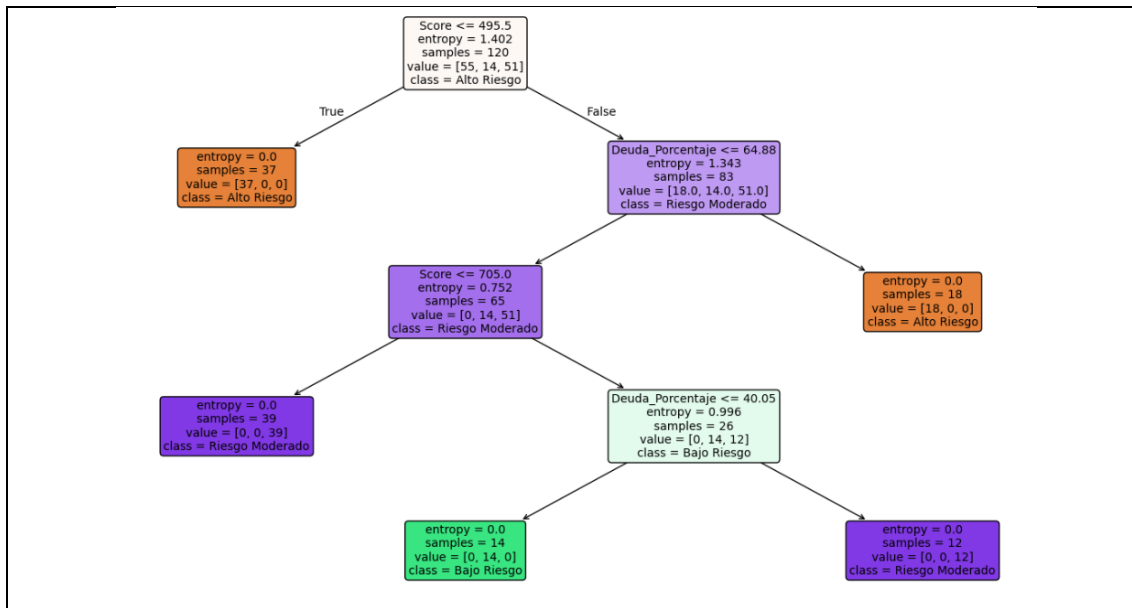
```

4. Copie los resultados de las métricas (numéricos y gráficos)

=== REPORTE DE CLASIFICACIÓN MULTICLASE ===

	precision	recall	f1-score	support	
Alto Riesgo	1.00	1.00	1.00	14	
Bajo Riesgo	1.00	1.00	1.00	4	
Riesgo Moderado	1.00	1.00	1.00	12	
accuracy			1.00	30	
macro avg	1.00	1.00	1.00	30	
weighted avg		1.00	1.00	1.00	30





5. Anote alguna conclusión relacionada con el proceso seguido y otra relacionada con los resultados obtenidos

- El proceso seguido y la importancia crítica de la preparación de datos al implementar la partición de datos estratificada (`stratify=y`), se garantizó que todas las categorías de riesgo (Bajo, Moderado y Alto) mantuvieran una proporción idéntica tanto en el conjunto de entrenamiento como en el de validación, evitando sesgos estadísticos. se limita la profundidad del árbol (`max_depth=4`) como técnica de pre-poda demostró ser un paso procedimental indispensable.
- Los resultados estadísticos obtenidos, evaluados mediante la matriz de confusión y el reporte de clasificación, demuestran la alta efectividad del Árbol de Decisión para resolver problemas financieros no lineales. El modelo no solo alcanzó niveles óptimos de precisión (*Accuracy* y *F1-score*)

5. Normalización (escalado) de datos numéricos (**Puntuación: 2 puntos**)

Complete la siguiente tabla, dados los datos observados (crudos) en la columna A (completar con valores al azar, de preferencia ordenados de menor a mayor), complete su valor normalizado en las columnas B, C, D, según se ha solicitado.

A Datos originales	B Normalización min-max	C Normalización Z-score	D Robust Scaling
1000	0	-1.04	-0.73
1200	0.05	-0.87	-0.57
1500	0.125	-0.63	-0.32
1800	0.2	-0.38	-0.08
2000	0.25	-0.21	0.08
2500	0.375	0.2	0.48
3100	0.53	0.69	0.979
5000	1	2.25	2.53

Responder además:

¿Cuáles son las razones principales para normalizar los datos, previa implementación de un modelo machine learning?

- Evitar sesgos
- Estabilidad en funciones de activación
- Mitigar outliers (datos atípicos)

Rúbrica de evaluación

Componente de aprendizaje:	Autónomo		Contacto con el Docente	X
Nombre de la Unidad:	Unidad 1: Fundamentos del aprendizaje automático y modelos supervisados Unidad 2: Modelos avanzados de clasificación y regresión			
Resultado(s) de aprendizaje:	- El estudiante comprenderá los fundamentos computacionales y matemáticos de Machine Learning (ML). - El estudiante aplicará modelos avanzados supervisados en problemas reales. - El estudiante aplicará principios de validación y métricas para evaluar modelos. - El estudiante analizará resultados con métricas multicriterio. - El estudiante aplicará mecanismos de normalización de datos.			
Nombre de la Actividad:	Evaluación de aplicación de la teoría del primer parcial (EAP1).			

Criterios de Evaluación	Escala de Valoración						
	Excelente (10 - 9,1)	Bueno (9 - 8,1)	Satisfactorio (8 - 7)	Necesita mejorar (6,9 - 0,1)	No entrega (0)	Puntaje	Comentarios (SIGEA)
1. Comprensión de ajuste de parámetros en un MLP	Los cálculos son precisos	Los cálculos son aceptables, con pequeñas imprecisiones.	Se detectan errores en los cálculos, o respuestas incompletas	Resultados imprecisos	No entrega	4	
2. Comprensión de la lógica de programación, para resolver problemas ML, uso de librerías Python, manejo de dataset, presentación de resultados.	Responde completamente sobre las imprecisiones planteadas en el código presentado. Interpreta adecuadamente la métrica y el resultado.	Identifica casi todas las imprecisiones planteadas en el código presentado, presenta un criterio aceptable con respecto a la métrica.	No logra detectar las imprecisiones o las respuestas son parcialmente correctas.	Respuestas no corresponden al planteamiento.	No entrega	4	
3. Resolución de problemas de regresión en el contexto del Machine Learning	Desarrolla el código / procedimiento de manera ordenada y reproducible. Responde a todas las interrogantes planteadas.	Desarrollo del código, responde las interrogantes de manera aceptable.	Desarrollo del código, sin embargo, las interrogantes no son respondidas correctamente, se encuentra incompleta o el criterio no es completamente adecuado.	Desarrolla parcialmente la respuesta.	No entrega	5	

4. Resolución de problemas de clasificación en el contexto del Machine Learning	Desarrolla el código / procedimiento de manera ordenada y reproducible. Responde a todas las interrogantes planteadas.	Desarrollo del código, responde las interrogantes de manera aceptable.	Desarrollo del código, sin embargo, las interrogantes no son respondidas correctamente, se encuentra incompleta o el criterio no es completamente adecuado.	Desarrolla parcialmente la respuesta.	No entrega	5	
5. Normalización de datos	Completa la actividad sin errores, presenta su reflexión de manera adecuada.	Completa la actividad y presenta su reflexión de manera aceptable.	Completa la actividad con pequeños errores o faltantes, el criterio es muy básico.	Completa la actividad con imprecisiones.	No entrega	2	

Puntaje total	20 puntos	_____
----------------------	------------------	--------------